

TEMPLATE PARA ENTREGA DO PROJETO DA DISCIPLINA

DevOps na Prática

Fase 1: Configuração e Automação Inicial

NOME DO ESTUDANTE

Patricia dos Santos Silva

SEÇÃO 1

Documentação de Planejamento

A) DESCRIÇÃO DO PROJETO, SEUS OBJETIVOS E REQUISITOS:

Descrição: TaskFlow API é uma API REST para gerenciamento de tarefas (to-do list), permitindo criar, organizar e acompanhar tarefas por status (a fazer, em andamento, concluída) e prioridade.

Objetivos: praticar um fluxo DevOps completo, da integração contínua ao provisionamento de infraestrutura como código, entregando uma API estável, testada automaticamente e com ambiente reproduzível via containers.

Requisitos funcionais: CRUD de tarefas e de usuários, autenticação via JWT e filtros de listagem por status e prioridade.

Requisitos não funcionais: cobertura de testes automatizados, pipeline de CI executado a cada push e infraestrutura provisionada de forma automatizada via Terraform.

B) PLANO DE INTEGRAÇÃO CONTÍNUA:

Ferramenta: GitHub Actions.

Gatilho: o pipeline é disparado a cada push e a cada pull request aberta contra a branch main.

Etapas do pipeline: (1) checkout do código; (2) instalação das dependências com `npm ci`; (3) lint com ESLint; (4) execução dos testes automatizados com Jest e geração do relatório de cobertura; (5) build da aplicação e da imagem Docker.

Critério de sucesso: o pipeline só é aprovado se lint, testes e build passarem sem erros; falhas bloqueiam o merge da pull request.

C) ESPECIFICAÇÃO DETALHADA DA INFRAESTRUTURA NECESSÁRIA:

Containers: aplicação e banco de dados executam em containers Docker, orquestrados localmente via Docker Compose.

Banco de dados: PostgreSQL, provisionado como instância gerenciada (Amazon RDS db.t3.micro).

Servidor de aplicação: instância EC2 (t3.micro) rodando a imagem Docker da API.

Rede: VPC (10.0.0.0/16) com subnet pública para a API e subnets privadas para o banco de dados, com security groups liberando apenas as portas necessárias (22, 80 e 443 na aplicação; 5432 no banco somente a partir do SG da aplicação).

Segredos e variáveis: credenciais e variáveis sensíveis gerenciadas via secrets do GitHub Actions e TF_VAR_*, nunca versionadas no repositório.

Provisionamento: 100% automatizado via Terraform, com os scripts versionados na pasta /infra do repositório.

SEÇÃO 2

Pipeline de Integração Contínua (CI)

A) CONFIGURAÇÃO DE UM REPOSITÓRIO DE CÓDIGO: **GitHub**

Insira o link de acesso ao repositório (GitHub).

Repositório criado no GitHub com estrutura organizada em pastas (src, test, infra e .github/workflows), README com instruções de execução do projeto, arquivo .gitignore e proteção configurada na branch main, exigindo pull request e checks de CI aprovados antes do merge.

Link do repositório: <https://github.com/PatriciaSSRS/taskflow-ap>

B) IMPLEMENTAÇÃO DE UM PIPELINE DE CI USANDO UM GITHUB ACTIONS:

Insira o link de acesso ao repositório (GitHub).

Pipeline implementado em `.github/workflows/ci.yml`, disparado a cada push e pull request contra a branch main. As etapas executadas são: checkout do código, instalação das dependências, lint com ESLint, execução dos testes automatizados com Jest (com relatório de cobertura, contra um serviço PostgreSQL) e build da imagem Docker da aplicação. O merge de uma pull request só é liberado se todas as etapas passarem com sucesso.

Link do repositório (com o workflow configurado): <https://github.com/PatriciaSSRS/taskflow-ap/blob/main/.github/workflows/ci.yml>

SEÇÃO 3

Scripts de Infraestrutura como Código

A) SCRIPTS PARA PROVISIONAMENTO DE INFRAESTRUTURA A PARTIR DA FERRAMENTA: Terraform

Insira o link de acesso ao repositório (GitHub).

Scripts Terraform para provisionar, na AWS, toda a infraestrutura necessária para rodar a API: instância EC2 para a aplicação containerizada, banco de dados RDS PostgreSQL, VPC com subnets pública e privadas, security groups liberando apenas as portas necessárias (22, 80 e 443) e um Elastic IP para acesso fixo à aplicação.

Link do repositório (pasta /infra): <https://github.com/PatriciaSSRS/taskflow-ap/tree/main/infra>

TESTES AUTOMATIZADOS IMPLEMENTADOS E INTEGRADOS:

Testes escritos com **Jest** e **Supertest**, separados em dois conjuntos: unit (utilitários de JWT, hash de senha e validação de dados) e integration (fluxo completo da API — autenticação, usuários e tarefas — contra um PostgreSQL real).

Os testes são **integrados ao pipeline de CI**: o job build-and-test executa `npm run test:coverage` a cada push e pull request, usando um container de serviço `postgres:16-alpine`. Uma falha de teste reprova o pipeline e bloqueia o merge.

Verificação local: 35 de 35 testes aprovados • cobertura de linhas $\approx 91\%$ • ESLint sem erros • `terraform validate` OK.

Link (pasta /test): <https://github.com/PatriciaSSRS/taskflow-ap/tree/main/test>